

CampusEats — Services, Contracts & Schema

Bhupesh Kumar & Aman Bali

August 2026

1. Capabilities

From the Assignment 1 brief, the distinct jobs the system does (the verbs), grouped into the capability an owning service will be responsible for:

#	Capability	Description
1	Register & Authenticate	Create accounts for students, restaurants and admins; log in / log out; verify identity and role for other services.
2	Browse & Search Catalogue	List restaurants, browse menus, search food items, view item detail.
3	Manage Menu	Restaurant adds, updates and removes food items and categories; opens/closes the restaurant.
4	Manage Cart	Add an item to the cart, remove an item, view cart contents, clear the cart at checkout.
5	Place Order	Convert a cart into a confirmed order against a specific restaurant and delivery address.
6	Track & Update Order Status	Student tracks order progress; restaurant accepts/rejects and advances order status; student cancels when permitted.
7	Make Payment	Authorize payment for an order, and process a refund on cancellation.
8	Manage Users & Restaurants (Admin)	Administrator manages user and restaurant accounts and system-wide data.
9	Send Notification	Notify a user about account, order or payment events.

That is 9 capabilities — comfortably above the minimum of 6.

2. Service Design

Capabilities were grouped by the data they naturally read and write, so that each resulting service owns a disjoint slice of the data (no two services share a table). This produced **6 services**:

Service	Owns (data)	Capabilities covered	Team member
Identity & Access Service	Users, Addresses	Register & Authenticate; Manage Users & Restaurants (account-level)	Bhupesh Kumar
Catalogue Service	Restaurants, Categories, FoodItems	Browse & Search Catalogue; Manage Menu	Bhupesh Kumar
Cart Service	Carts, CartItems	Manage Cart	Aman Bali
Order Service	Orders, OrderItems, OrderStatusHistory	Place Order; Track & Update Order Status	Bhupesh Kumar
Payment Service	Payments, PaymentTransactions	Make Payment	Aman Bali
Notification Service	Notifications	Send Notification	Aman Bali

Team: Bhupesh Kumar and Aman Bali. Bhupesh owns the identity/catalogue/ordering path (Identity & Access, Catalogue, Order); Aman owns cart, payment and notification (Cart, Payment, Notification) — each service has exactly one clearly identified owner, as the task requires.

The full **service design diagram** (boxes = services labelled with the data they own; arrows = cross-service calls, labelled with the operation used) is submitted as `services.drawio` (editable source) and `services.png` (exported). The diagram is reproduced below for reference.

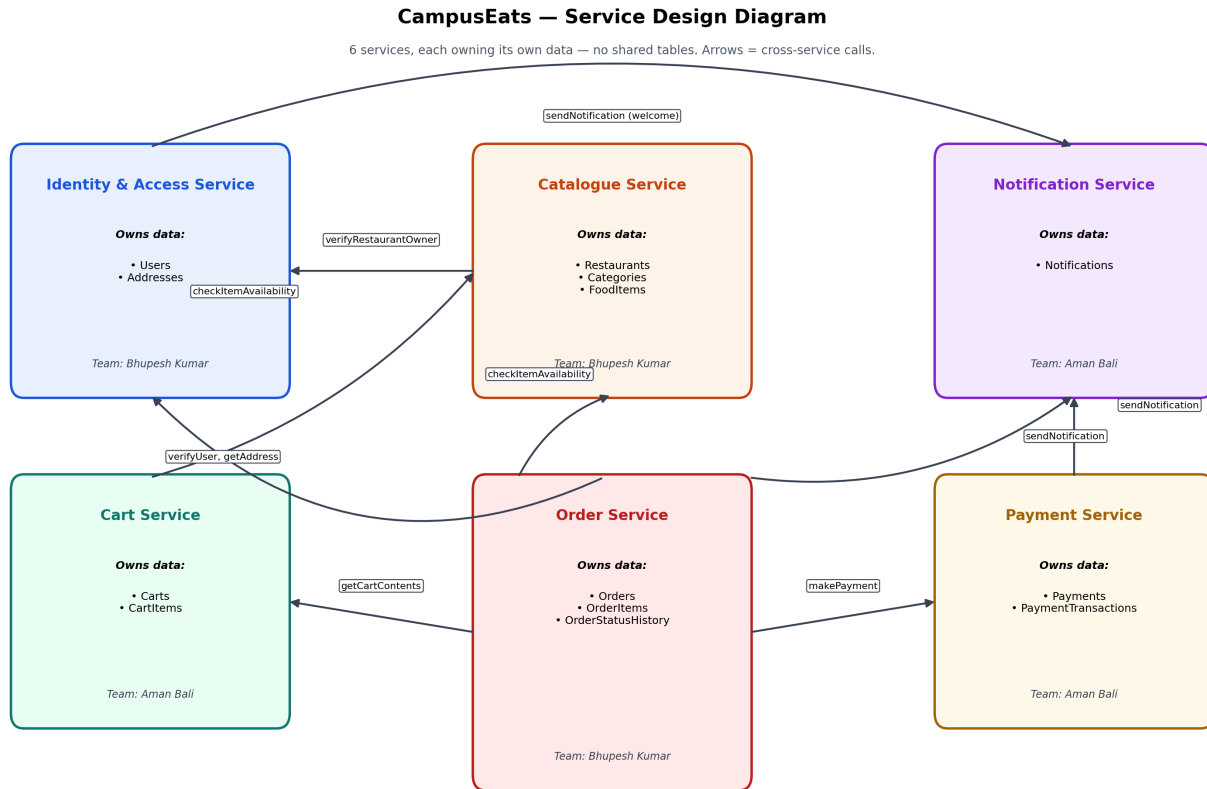


Figure 1: CampusEats service design diagram

Boundary rule check: every table appears in exactly one service’s “Owns” list above (see the Database Schema in Section 5) — no table is shared between two services. Any place a service needs another service’s data, it holds an opaque reference (`*_ref`) and calls that service’s contract, rather than reading its tables directly.

3. Service Contracts

Each table below lists the operations a service exposes to **other services** (its public contract). Internal details — how data is stored, identifier formats, SQL, or private helper functions — are deliberately not part of the contract; only the operation is exposed.

3.1 Identity & Access Service

Operation	Input	Output	Errors
<code>registerUser</code>	role, name, email, password, phone	userRef, role	<code>EmailAlreadyRegistered</code> , <code>InvalidInput</code>
<code>login</code>	email, password	sessionToken, userRef, role	<code>InvalidCredentials</code> , <code>AccountLocked</code>
<code>verifyUser</code>	sessionToken <i>or</i> userRef	userRef, role, isActive	<code>InvalidSession</code> , <code>SessionExpired</code> , <code>UserNotFound</code>
<code>getDeliveryAddress</code>	userRef, addressLabel?	address (building, room/landmark, campus zone)	<code>AddressNotFound</code> , <code>Unauthorized</code>

3.2 Catalogue Service (*main service*)

Operation	Input	Output	Errors
<code>browseRestaurants</code>	campusZone?, category?	list of restaurant summaries	<code>NoResultsFound</code>
<code>searchFoodItems</code>	query, filters?	list of food item summaries	<code>InvalidQuery</code>
<code>checkItemAvailability</code>	list of itemRefs (+ quantities)	per-item availability flag and current unit price	<code>ItemNotFound</code> , <code>RestaurantClosed</code>
<code>manageMenuItem</code>	restaurantRef, action (add/update/remove), item details	itemRef, status	<code>Unauthorized</code> , <code>InvalidItemData</code>

3.3 Cart Service

Operation	Input	Output	Errors
<code>addToCart</code>	userRef, itemRef, quantity	cartRef, updatedSubtotal	<code>ItemUnavailable</code> , <code>InvalidQuantity</code>
<code>removeFromCart</code>	userRef, itemRef	cartRef, updatedSubtotal	<code>ItemNotInCart</code>
<code>getCartContents</code>	userRef <i>or</i> cartRef	list of cart items, subtotal	<code>CartEmpty</code> , <code>CartNotFound</code>

3.4 Order Service (*main service*)

Operation	Input	Output	Errors
placeOrder	userRef, cartRef, deliveryAddress, paymentMethod	orderRef, status, estimatedDeliveryTime, totalAmount	CartEmpty, ItemUnavailable, RestaurantUnavailable, PriceMismatch, PaymentFailed, InvalidAddress, Unauthorized, DuplicateOrder
cancelOrder	orderRef, userRef	orderRef, status	OrderNotCancellable, OrderNotFound, Unauthorized
trackOrder	orderRef	status, statusHistory, estimatedDeliveryTime	OrderNotFound
updateOrderStatus	orderRef, restaurantRef, newStatus	orderRef, status	InvalidStatusTransition, Unauthorized, OrderNotFound

3.5 Payment Service

Operation	Input	Output	Errors
makePayment	orderRef, amount, method	paymentRef, status	PaymentDeclined, InvalidAmount, GatewayTimeout
getPaymentStatus	paymentRef <i>or</i> orderRef	status	PaymentNotFound
refundPayment	paymentRef, amount	refundRef, status	RefundNotAllowed, PaymentNotFound

3.6 Notification Service

Operation	Input	Output	Errors
sendNotification	userRef, type, message	notificationRef, status	DeliveryFailed, InvalidUser

(Catalogue and Order are treated as the two main services and each expose 4 operations; every other service exposes at least one, satisfying the ≥ 3 / ≥ 1 requirement.)

4. Central Operation: placeOrder

placeOrder is CampusEats' central operation — it is where the Cart, Catalogue, Identity, Payment and Notification services all meet through the Order Service.

Inputs

- **userRef** — opaque reference to the authenticated student placing the order
- **cartRef** — opaque reference to that student's current cart
- **deliveryAddress** — structured delivery address (building/hostel, room or landmark, campus zone)
- **paymentMethod** — the chosen payment option (wallet / card / UPI)

Outputs (success)

- **orderRef** — opaque reference to the newly created order

- **status** — initial order status (e.g. **placed**)
- **estimatedDeliveryTime** — an estimated delivery window
- **totalAmount** — the computed total charged (items + any fees)

Error cases

- **CartEmpty** — the cart has no items
- **ItemUnavailable** — one or more items are no longer available at the restaurant
- **RestaurantUnavailable** — the restaurant is currently closed / not accepting orders
- **PriceMismatch** — the price cached in the cart no longer matches the Catalogue Service's current price (re-validation failure)
- **PaymentFailed** — the payment could not be authorized
- **InvalidAddress** — the delivery address falls outside a supported campus zone
- **Unauthorized** — the caller's session is invalid or expired
- **DuplicateOrder** — a retried/duplicate submission was detected (idempotency check)

Internal details **placeOrder** must hide from callers

- How order and order-item data is persisted, and any internal identifier/primary-key formats
- The exact internal call sequence (Cart lookup → Catalogue re-validation → Identity verification → Payment authorization → Order persistence → Notification dispatch) and each step's retry/timeout policy
- The consistency strategy used to keep Order, Payment and Cart in agreement if a step fails partway through (e.g. a saga/compensation flow that releases a reserved item if payment fails)
- The pricing, delivery-fee, tax and estimated-time calculations
- Which downstream calls are synchronous (must complete before responding, e.g. payment authorization) versus asynchronous/fire-and-forget (e.g. sending the confirmation notification)
- Any locking or concurrency control used to prevent two students over-ordering the last unit of an item
- Any caching used to speed up availability checks

A caller of **placeOrder** only ever sees the inputs, the outputs, and the named error cases above — never how the Order Service achieves them internally.

5. Database Schema

Each service's tables hold only the data it owns; no table belongs to two services. Cross-service relationships are represented as **opaque reference columns** (e.g. **user_ref**, **item_ref**) rather than foreign keys, because a foreign key would require a shared database and would violate the service boundary. Foreign keys are used only **within** a service, between its own tables.

The full **ER diagram** is submitted as **schema.drawio** (editable source) and **schema.png** (exported), and the equivalent CREATE TABLE statements are submitted as **schema.sql**. The diagram is reproduced below.

Table ownership summary

Service	Tables
Identity & Access Service	Users, Addresses
Catalogue Service	Restaurants, Categories, FoodItems
Cart Service	Carts, CartItems
Order Service	Orders, OrderItems, OrderStatusHistory
Payment Service	Payments, PaymentTransactions
Notification Service	Notifications

6. Service Validation

Each service is checked against the five properties of a Service: **reachable**, **self-contained**, **has a contract**, **independent**, **loosely coupled**.

CampusEats — Database Schema (ER Diagram)

Each service owns its own tables. Cross-service references (*_ref) are opaque values, not foreign keys.

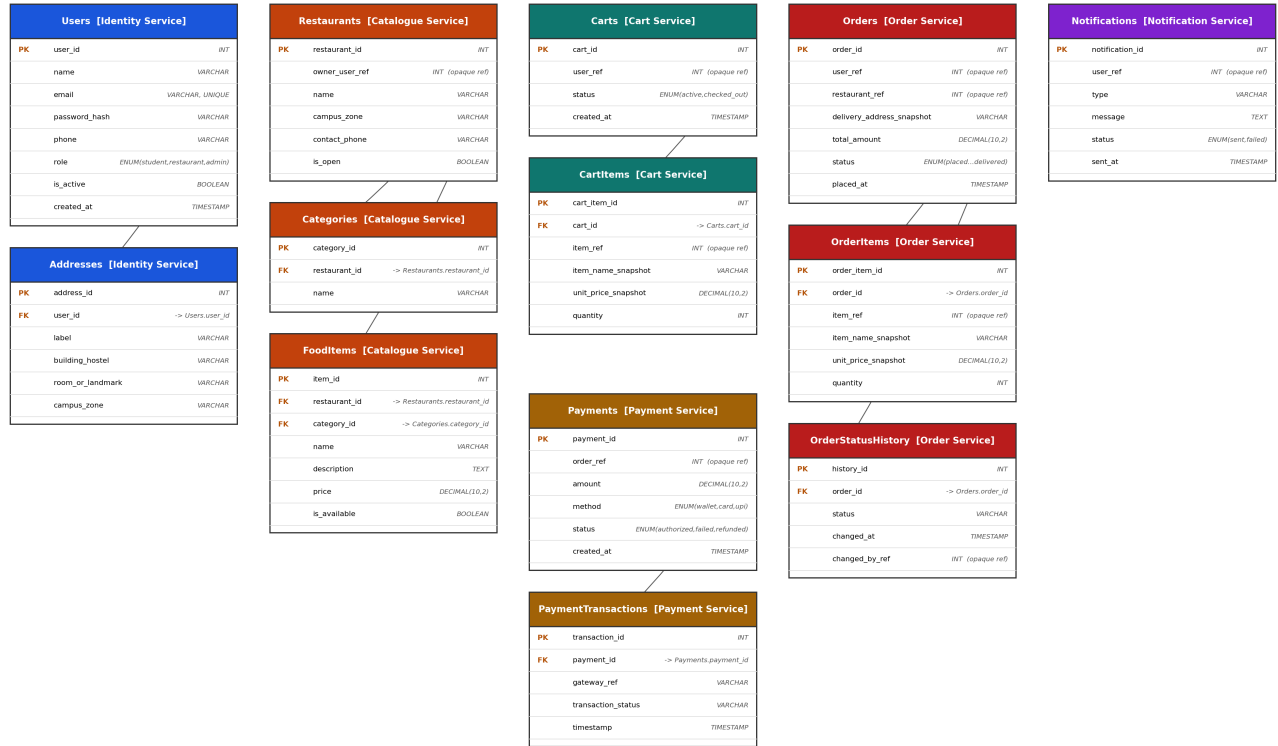


Figure 2: CampusEats database schema (ER diagram)

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled
Identity & Access	Pass	Pass	Pass (3.1)	Pass	Pass
Catalogue	Pass	Pass	Pass (3.2)	Pass	Partial (note 1)
Cart	Pass	Pass	Pass (3.3)	Pass	Pass
Order	Pass	Pass	Pass (3.4)	Partial (note 2)	Partial (note 2)
Payment	Pass	Pass	Pass (3.5)	Pass	Pass
Notification	Pass	Pass	Pass (3.6)	Pass	Pass

Basis for the Pass columns. *Reachable* – every service exposes callable operations over a network boundary (Section 3). *Self-contained* – each owns a disjoint set of tables it alone reads and writes (Section 5). *Has a contract* – every service has a documented operation table with input/output/errors (Section 3). *Independent* – each service’s own logic and data can be changed or redeployed without touching another service’s internals, except where noted below.

Note 1 – Catalogue’s loose coupling is partial. `checkItemAvailability` is called synchronously by both the Cart and the Order Service, so a Catalogue outage would stall both of them. **Fix:** give Cart and Order a short-lived local cache of price/availability so a brief Catalogue outage degrades gracefully instead of blocking every add-to-cart and checkout.

Note 2 – Order fails independence and loose coupling. `placeOrder` synchronously calls four other services (Cart, Catalogue, Identity, Payment) within a single request, so the Order Service’s own availability is coupled to all four of theirs, and it cannot be tested or deployed fully on its own. **Fix:** keep only the payment authorization call synchronous (it must succeed before an order is confirmed); make the notification dispatch fully asynchronous; and wrap the remaining multi-service write in a saga with compensating actions (e.g. releasing a reserved item if payment fails) instead of one long synchronous transaction, so a slow or failed downstream service degrades rather than hard-failing the whole operation.

Overall: four of the six services pass all five properties cleanly. Catalogue and, especially, Order carry coupling risk because `placeOrder` is CampusEats’ central, multi-service operation by design – the fixes noted above keep that coupling manageable without merging services or sharing data.